# Creating a Plugin Page

🔃 Information last updated 22 sept 2025

| SME | Jan Rusch |
| --- | --- |
| EPIC | CIC Custom User Interface and Custom Development |



## 🟦 Instructions

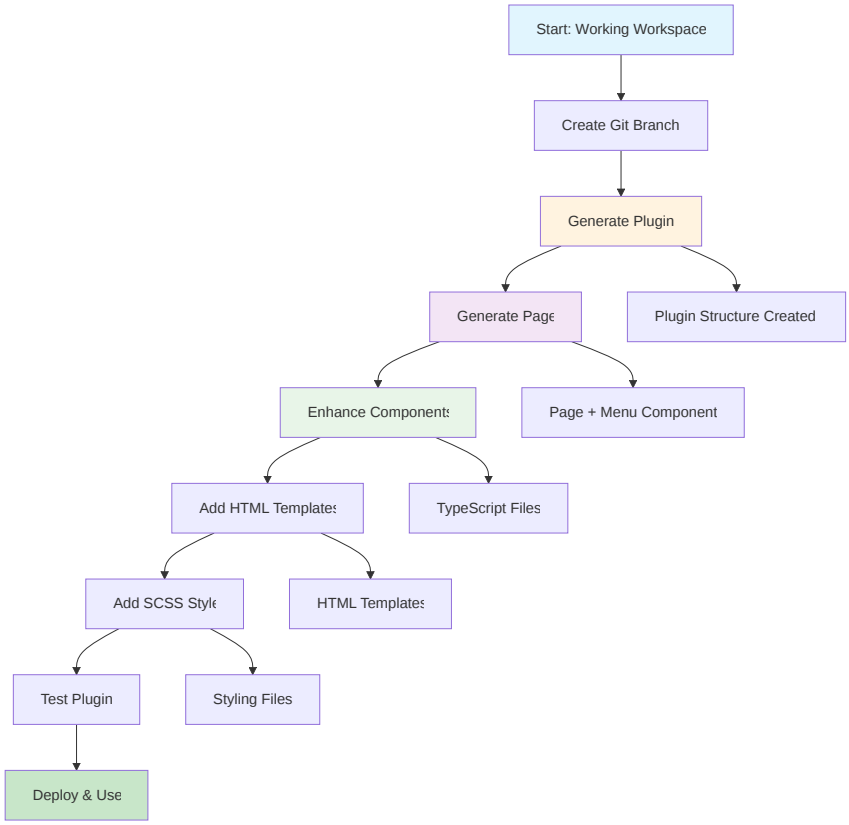
Hey developers! Welcome back to another Hyland Experience tutorial. Today, I'm going to show you how to create custom plugin pages that extend your Hyland Experience Workspace with new functionality. This is where the real customization magic happens!

**What you'll learn today:**

- How to create a plugin.
- How to generate custom pages with navigation
- How to properly structure your components
- Best practices for plugin development

**Let's build something awesome together!**

## 📊 Plugin Development Process Flow




---

## 📋 Prerequisites

Before we start, make sure you have:

- **A working Hyland Experience Workspace** (check out Creating an Hyland Experience Application tutorial if you haven't set this up yet)
- **Node.js 18+ and npm** installed
- **A code editor** (VS Code recommended)
- **Git** for version control (highly recommended)

---

## 🚀 Step 1: Create a New Git Branch

**IMPORTANT**: Before we start generating code, create a new branch! The plugin generation commands create and modify many files, so having a clean branch makes it easy to start over if needed.

```
git checkout -b feature/my-custom-plugin
```

> 🤓 *Trust me on this one - you'll thank me later when you want to experiment with different plugin names!*

---

## 🔧 Step 2: Generate Your Plugin

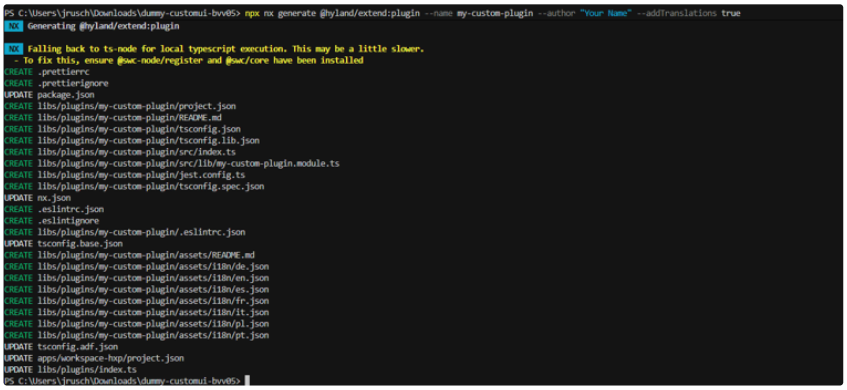
The plugin is the foundation that will contain all your custom pages and widgets. Let's create one:

```
npx nx generate @hyland/extend:plugin --name my-custom-plugin --author "Your Name" --addTranslations true
```

**Let me break down these parameters:**

- `--name my-custom-plugin`: This becomes your plugin identifier
- `--author "Your Name"`: Your name (though it doesn't appear in generated files)
- `--addTranslations true`: Enables internationalization support

> *Choose a descriptive name that reflects your plugin's purpose. Use kebab-case (lowercase with hyphens).*




**What this command does:**

- Creates a new plugin library in `libs/plugins/my-custom-plugin/`
- Sets up the basic plugin structure
- Configures translation files if enabled
- Updates the necessary configuration files

---

### 📄 Step 3: Generate a Page

Now let's create a page within your plugin. A page consists of two components:

1. **Main page component** - displays in the main content area
2. **Menu item component** - adds a navigation link to the sidebar

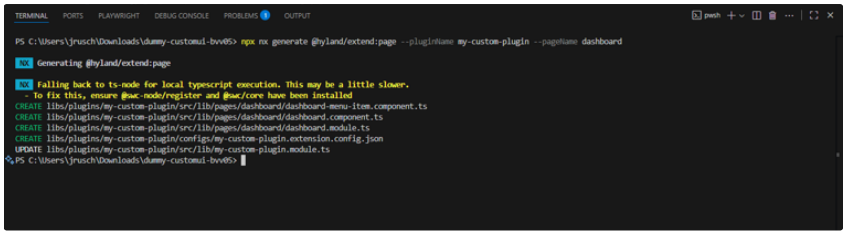
```
npx nx generate @hyland/extend:page --pluginName my-custom-plugin --pageName dashboard
```

**Parameters explained:**

- `--pluginName my-custom-plugin`: The name of your plugin (without "plugins-" prefix)
- `--pageName dashboard`: The name of your page (will be converted to Angular naming conventions)

**What gets generated:**

- `dashboard.component.ts` - Main page component
- `dashboard-menu-item.component.ts` - Sidebar navigation component
- Both components are properly wired into your plugin

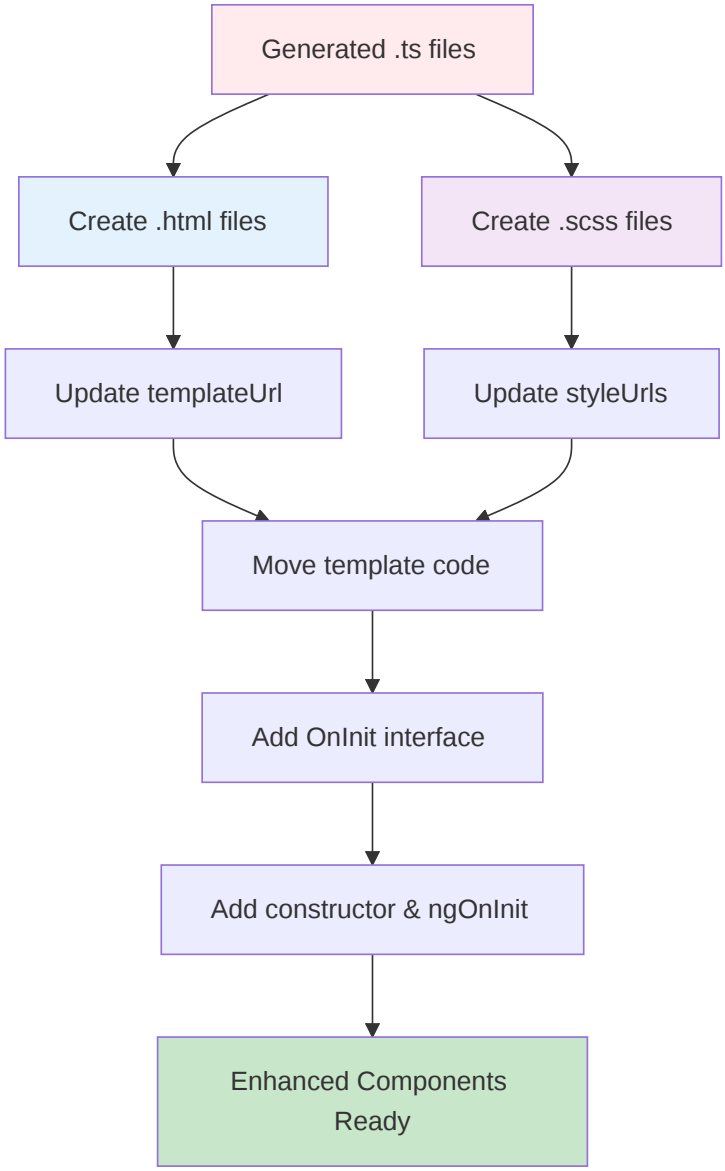



---

### 🎨 Step 4: Enhance Your Components (Recommended)

The generated components only create TypeScript files. For better development experience, let's add HTML templates and SCSS files:

**🔧 Component Enhancement Process:**



#### 4.1 Create Template and Style Files

Navigate to your page directory (usually `libs/plugins/my-custom-plugin/src/lib/pages/dashboard/`) and create these files:

```
dashboard.component.html
dashboard.component.scss
```

dashboard.component.html:

```
my-custom-plugin-dashboard Works! {{ 'PLUGIN_MESSAGE' | translate }}
```

The dashboard.component.scss keeps stay empty for futher use.

#### 4.2 Update Component Files

**For the main page component (`dashboard.component.ts`):**

```
import { Component } from '@angular/core';
import { TranslateModule } from '@ngx-translate/core';

@Component({
  selector: 'plugins-my-custom-plugin-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.scss'],
  imports: [TranslateModule],
  standalone: true,
})
export class DashboardComponent {
}
```

---

### 🚀 Step 5: Test Your Plugin

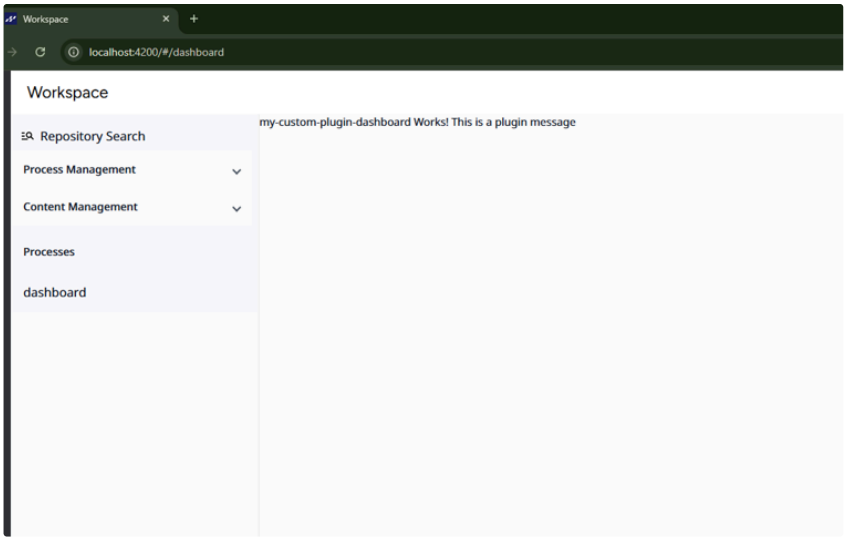
Now let's see your plugin in action:

1. **Start your development server:**

   ```
   npm start workspace-hxp
   ```

2. **Navigate to your workspace** at `<http://localhost:4200>`
3. **Look for your new menu item** in the sidebar navigation
4. **Click on it** to see your custom page load in the main content area

*If everything worked correctly, you should see your custom dashboard page with the content you created!*



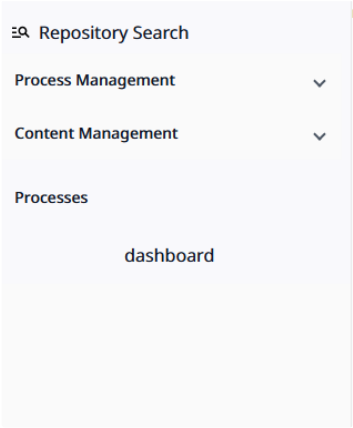

---

🎨 **Step 6: Customizing Your Menu Item**

The generated menu item component is basic. Let's enhance it:

**Update your** `dashboard-menu-item.component.ts`:

With the Default Styling we get the dashboard Center on the Sidebar.




Therefor i like to add the styling to the Navigation Button:

```
padding: 0px 16px;
justify-content: flex-start;
```

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { MatButtonModule } from '@angular/material/button';

@Component({
    template: `
        <button
            mat-button
            (click)="navigateToPage()"
            style="
```

```
                    width: 100%;
                    font-family: 'Noto Sans';
                    font-size: 16px;
                    font-weight: normal;
                    padding: 0px 16px;
                    justify-content: flex-start;
                "
        >
        <span
            style="
                display: block;
                width: 100%;
                text-align: left;
            ">dashboard</span>
    </button>`,
    standalone: true,
    imports: [MatButtonModule],
})
export class DashboardMenuItemComponent {
    constructor(
        private router: Router,
    ) {}

    navigateToPage(): void {
        void this.router.navigate(['/dashboard']);
    }
}

```

🔧 **Adding More Pages?**

Want to add additional sidebar items? Here's how:

**1. Generate another page:**

```
npx nx generate @hyland/extend:page --pluginName my-custom-plugin --pageName reports
```

**2. The system automatically:**

- Creates `reports.component.ts` and `reports-menu-item.component.ts`
- Registers them in your plugin module
- Updates the extension configuration
- Adds the route

**3. Customize the new menu item:**

```
// In reports-menu-item.component.ts
export class ReportsMenuItemComponent {
    navigateToPage(): void {
        void this.router.navigate(['/reports']); // Different route
    }
}
```

🎯 **Pro Tips for Menu Items**

**Styling:**

- Keep consistent with your workspace theme
- Use hover states for better UX

- **Navigation:**
- Always use Angular Router for navigation
- Use relative paths when possible
- Handle navigation errors gracefully

### 🐛 Troubleshooting Menu Items

**Menu item not appearing?**

- ✅ Check component registration in module
- ✅ Verify extension configuration
- ✅ Ensure component is imported

**Navigation not working?**

- ✅ Check route configuration
- ✅ Verify route path matches navigation call
- ✅ Ensure RouterModule is imported

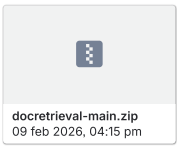
**Styling issues?**

- ✅ Check CSS class names
- ✅ Verify Angular Material imports
- ✅ Use browser dev tools to inspect

---

### Video: Creating a Custom UI Page

Panopto




### 💬 Questions?

Reach out to the team! We're here to help you! 🚀

Thanks, and happy coding!

## 📋 Related articles